DICTAMEN TÉCNICO: RESPUESTA OFICIAL A AUDITORÍA

Evaluación Crítica, Contraste de Evidencias y Plan de Remediación Definitivo

Respuesta técnica integral y exhaustiva a los informes de seguridad y administración DBA remitidos por Samir sobre la plataforma Mundo Escolar.

PROYECTO EVALUADO
Librería y Papelería Mundo Escolar (ERP, POS & DBA)

MOTOR & TECNOLOGÍAS
Microsoft SQL Server 2022 (RTM) • NestJS 11 • React 19

DOCUMENTOS RESPONDIDOS
1. Obs. de Seguridad (10 págs) • 2. Seguridad DBA (8 págs)

RAMA & COMMIT BASE
`development` / `Feature-Mau` (Commits `4b920ec` y `6c7e03d`)

VEREDICTO DE INGENIERÍA
Aceptación Total de Hallazgos Críticos • Remediación en Curso

FECHA DE EMISIÓN
Noviembre de 2026

Declaración de Postura Técnica y Honestidad Profesional

Tras una inspección línea por línea del código fuente y de la base de datos real, **este dictamen ratifica que las observaciones técnicas formuladas por Samir en ambos informes son plenamente acertadas y fundadas**. Se desestima cualquier pretensión de sostener una "certificación de seguridad plena" previa sobre los respaldos. La presencia de SQL dinámico concatenado en procedimientos `WITH EXECUTE AS OWNER` combinada con `TRUSTWORTHY ON` y el usuario `sa` constituía un vector crítico de elevación de privilegios hacia `sysadmin` que no admitía defensa técnica. Este documento detalla la aceptación sin reservas de los hallazgos válidos, el contraste objetivo de pruebas y la arquitectura de solución definitiva.

1. Resumen Ejecutivo y Dictamen de Ingeniería

La seguridad en sistemas de información no se fundamenta en intenciones ni en la mera existencia de pantallas de administración; se valida a través del aislamiento riguroso de contextos de ejecución, el Principio de Menor Privilegio (PoLP) y la ausencia comprobada de vectores de inyección. La auditoría externa realizada por Samir desarticuló una falsa sensación de seguridad generada en la certificación previa, aportando un examen de código exhaustivo, riguroso y constructivo.

> **Veredicto de Seguridad Global:** El proyecto Mundo Escolar cuenta con una arquitectura de respaldos operativamente funcional y verificada en disco (capacidad real de emitir backups Full/Diff/Log, scripts DRP y mantenimiento de índices), pero su esquema de permisos e invocación contenía **debilidades de severidad crítica** que impedían certificarlo como seguro. Se asume el compromiso de corregir cada vector antes de cualquier sustentación académica ante el profesor Torrentes.

2. Desmontaje Técnico del Vector Crítico de Inyección (Hallazgo P0)

El hallazgo más contundente del informe de Samir radica en la interacción fatal entre cuatro componentes del sistema:

```
Ruta libre en API → Concatenación T-SQL → EXECUTE AS OWNER → Dueño sa → TRUSTWORTHY ON = RCE / sysadmin
```

2.1. Inspección del Código Vulnerable en Procedimientos Almacenados

En el archivo `sql/01_despliegue_secuencial/10_CrearProcedimientos.sql` (líneas 2711 a 2720), el procedimiento de verificación de copias fue implementado de la siguiente manera:

```
-- Procedimiento 72: SP_DBA_VerificarIntegridadBackup
CREATE OR ALTER PROCEDURE dbo.SP_DBA_VerificarIntegridadBackup
    @RutaArchivoBak VARCHAR(500)
WITH EXECUTE AS OWNER  -- Se ejecuta bajo el contexto del propietario de la BD ('sa')
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        -- FALLA VULNERABLE: Concatenación directa de cadenas sin escapar comillas simples
        DECLARE @Sql NVARCHAR(MAX) = N'RESTORE VERIFYONLY FROM DISK = ''' + @RutaArchivoBak + ''';';
        EXEC sp_executesql @Sql;
...
```

2.2. Por qué la concatenación derivaba en el control total del servidor

1. **Ruta no saneada desde la API:** En `dba.dto.ts`, la propiedad `rutaArchivoBak` se validaba únicamente con `@IsString()` y `@MaxLength(500)`. Permitía la inyección de comillas simples (`'`), comandos T-SQL arbitrarios y rutas UNC de red (`\hostshare`).
2. **Suplantación de Identidad Elevada:** La cláusula `WITH EXECUTE AS OWNER` fuerza al motor a ejecutar el lote dinámico con las credenciales del dueño de la base de datos. En `01_CrearBaseDatos.sql` (línea 80), el dueño explícito es el superusuario `sa`.
3. **Ruptura del Sandbox mediante TRUSTWORTHY ON:** En SQL Server, una base de datos normal no puede tocar recursos fuera de sí misma aunque corra como `sa` dentro del catálogo. Sin embargo, en la línea 84 de `01_CrearBaseDatos.sql` se configuró `ALTER DATABASE MUNDO_ESCOLAR SET TRUSTWORTHY ON;`. Esta directiva le ordena a SQL Server confiar plenamente en los ensamblados y procedimientos de la base, permitiendo que la sesión salte la frontera de la base de datos hacia el ámbito de servidor con privilegios de `sysadmin`.
4. **Impacto de Rutas UNC:** Un atacante que suministre `\192.168.1.50share␀ackup.bak` obliga al servicio SQL Server de Windows a emitir una negociación SMB saliente hacia la IP del atacante, exponiendo el hash criptográfico NetNTLMv2 de la cuenta de servicio de Windows para su captura y cracking (*SMB Relay / Hash Capture*).

> **Conclusión del punto crítico:** Samir tiene 100% de razón. Nuestra afirmación previa de que *"la inyección SQL no permitiría respaldos libres ni exfiltración"* carecía de sustento técnico. El vector es reproducible, real y de impacto crítico.

3. Respuesta Detallada a los Hallazgos de Seguridad (S01 a S05)

A continuación se expone la respuesta técnica, análisis de código y acción de remediación comprometida para cada una de las primeras cinco observaciones del informe general de Samir:

ID	SEVERIDAD	HALLAZGO AUDITADO	EVIDENCIA Y DIAGNÓSTICO REAL	DICTAMEN Y REMEDIACIÓN
S01	CRÍTICA	**Contraseña predeterminada en clientes** Asignación de clave fija cuando el cliente no la envía.	En `clientes.service.ts` líneas 125 y 174: `const passPlano = (dto.contrasena && dto.contrasena.trim()) || 'Cliente2026!';` Los DTOs definían `contrasena` opcional.	**100% PROCEDENTE** Vulnerabilidad de lógica severa. Permite a cualquiera acceder a cuentas de clientes recién creadas. Se exigirá contraseña obligatoria con mínimo 8 caracteres y política de complejidad.
S02	CRÍTICA	**Credenciales SQL en el repositorio** Contraseñas de logins en scripts versionados en Git.	En `Seguridad.sql` líneas 21 a 33: `CREATE LOGIN login_mundo_admin WITH PASSWORD = N'AdminSecure2026!'` Se definieron 5 contraseñas en texto claro.	**100% PROCEDENTE** Es una mala práctica crítica mantener secretos duros en Git. Se rotarán las claves en las instancias locales y se convertirá el script a un modelo parametrizado por variables de entorno.
S03	ALTA	**Hash SHA-256 plano sin sal** Almacenamiento de contraseñas con función criptográfica rápida.	En `password.util.ts` línea 11: `createHash('sha256').update(textoPlano, 'latin1').digest('hex')` No existe salt individual ni función KDF lenta.	**100% PROCEDENTE** OWASP prohíbe SHA-256 sin sal. Se diseñará un plan de migración hacia Argon2id / bcrypt, implementando prefijos de versión de hash para actualizar usuarios al iniciar sesión.

S04	ALTA	**TRUSTWORTHY ON y sa**
		Configuración de base de datos que anula el aislamiento de seguridad.
			En `01_CrearBaseDatos.sql` (líneas 80-84):
			`ALTER AUTHORIZATION ... TO sa;`
			`ALTER DATABASE ... SET TRUSTWORTHY ON;`
				100% PROCEDENTE
				Se retirará `TRUSTWORTHY ON` de todos los scripts. Para los procedimientos de mantenimiento que requieran permisos fuera de la BD, se utilizará firma por certificados o permisos PoLP.

S05	ALTA	**Privilegios excesivos en login de app**
		`usr_mundo_app` con permisos amplios y backupoperator.
			En `Seguridad.sql` (líneas 80-91):
			`ALTER ROLE db_backupoperator ADD MEMBER usr_mundo_app;`
			`GRANT EXECUTE ON SCHEMA::dbo TO rol_db_aplicacion;`
				100% PROCEDENTE
				El usuario común de la API no debe ser operador de respaldos ni tener acceso irrestricto a todo el esquema. Se revocarán los permisos globales y se concederá EXECUTE únicamente a SPs requeridos.

3.1. Profundización en el Hallazgo S01 (Lógica de Autenticación de Clientes)

El análisis de Samir demostró que en el flujo de auto-registro del portal de clientes, cualquier usuario anónimo podía registrarse en el endpoint público `POST /clientes/particular` sin proveer una clave. El backend, en lugar de rechazar la solicitud por incompleta, le asignaba en silencio la cadena fija `Cliente2026!`.

Dado que el nombre de usuario se genera de forma predecible como `cli.<cédula>`, cualquier atacante que conozca el número de cédula de un cliente registrado bajo esta modalidad podía autenticarse en el portal institucional, ver su historial de compras, saldos de crédito y datos personales. **La gravedad calificada como Crítica por Samir es indiscutible.**

3.2. Profundización en el Hallazgo S03 (Algoritmo de Derivación de Claves)

La justificación histórica en el proyecto para usar `HASHBYTES('SHA2_256', ...)` radicaba en mantener paridad directa con los procedimientos almacenados de SQL Server creados en el Hito 1. No obstante, desde la perspectiva de seguridad moderna:

- SHA-256 es una función de resumen diseñada para integridad de archivos, no para almacenamiento de secretos humanos. Un clúster moderno de GPUs puede calcular más de 10,000 millones de hashes SHA-256 por segundo.
- Al no incorporar un **Salt** (valor aleatorio único por usuario), dos usuarios con la misma contraseña producen exactamente el mismo hash hexadecimal, permitiendo ataques de colisión e identificación de patrones masivos mediante tablas Rainbow precalculadas.

4. Respuesta Detallada a los Hallazgos de Seguridad (S06 a S10)

ID	SEVERIDAD	HALLAZGO AUDITADO	EVIDENCIA Y DIAGNÓSTICO REAL	DICTAMEN Y REMEDIACIÓN

S06	MEDIA-ALTA	**Cifrado y certificados en conexión SQL**
		Valores predeterminados inseguros en la configuración del backend.
			En `configuration.ts` líneas 8-11:
			`user: process.env.DB_USER || 'sa'`
			`encrypt: ... === 'true' // Default: false`
			`trustServerCertificate: ... !== 'false'`
				100% PROCEDENTE
				Tener a `sa` como usuario de fallback promueve malas prácticas en despliegues donde no se configure el `.env`. Se forzará `DB_USER` explícito y advertencia en consola.

S07	MEDIA	**Ausencia de Rate Limiting observable** Sin protección contra ataques de fuerza bruta o DoS en API.	`package.json` y `app.module.ts` carecen de `@nestjs/throttler`. Los endpoints `/auth/login` y `/clientes/*` admiten peticiones ilimitadas.	**100% PROCEDENTE** Cualquier script automatizado puede probar diccionarios de claves sin bloqueo. Se añadirá `ThrottlerModule` con límites de 10 peticiones/minuto en rutas de autenticación.
S08	MEDIA	**JWT persistido en localStorage** Exposición de tokens de sesión ante vulnerabilidades XSS.	En `AuthContext.jsx` y `client.js`: `localStorage.setItem('token', token)` Cualquier script con acceso al origen puede leerlo.	**PARCIALMENTE PROCEDENTE** Es el patrón común en SPAs académicas sin backend BFF, pero Samir acierta en que ante un XSS la sesión es secuestrable. Se evaluará CSP y expiración corta de tokens.
S09	MEDIA	**Divulgación de errores internos de SQL** Mensajes de motor SQL devueltos en respuestas HTTP.	En `sql-exception.filter.ts` línea 44: `mensaje = 'Error de SQL Server [' + errorSQL + ']: ' + texto;` En `sql.service.ts` línea 213: `err.message` en `/health`.	**100% PROCEDENTE** El cliente HTTP no debe recibir nombres de tablas ni números de error interno de SQL Server. Se sanitizarán las salidas con códigos de correlación en logs protegidos.
S10	BAJA-MEDIA	**Swagger y diagnóstico públicos** Enumeración de API y detalles de SQL expuestos sin autenticación.	En `main.ts` y `app.controller.ts`: `/api/docs` público sin guardia. `/health` devuelve recuento de tablas OLTP, DW y versión del motor SQL.	**100% PROCEDENTE** Facilita el reconocimiento de huella técnica a terceros. Se deshabilitará Swagger en producción y `/health` solo devolverá estado booleano de disponibilidad.

4.1. Análisis del Manejador de Excepciones SQL (Hallazgo S09)

En la revisión del filtro `SqlExceptionFilter` se detectó que, si bien el filtro mapea con acierto los códigos 2627 (violación de PK/UNIQUE) y 547 (violación de Foreign Key) a mensajes de negocio, cualquier otro código de SQL Server (como el error 515 de valores nulos o errores de sintaxis dinámica) se concatenaba en la respuesta JSON enviada al cliente web:

```
// Código observado en sql-exception.filter.ts
mensaje = `Error de SQL Server [${errorSQL}]: ${texto}`;
```

Esto significa que si un atacante envía datos maliciosos, la base de datos responde revelando la estructura de la tabla, los nombres exactos de las columnas y el nombre de la instancia. **Este comportamiento fue corregido conceptualmente en el plan de remediación para que solo devuelva un UUID de trazabilidad** y el detalle se conserve exclusivamente en el archivo de registro del servidor.

4.2. Análisis del Endpoint de Monitoreo /health (Hallazgo S10)

El servicio `sql.service.ts` ejecutaba una consulta contra las vistas del sistema (`sys.tables`, `sys.procedures`, `SERVERPROPERTY('ProductVersion')`). Si bien esto proporcionaba un panel de estado visual muy atractivo para las demostraciones locales, en un despliegue expuesto divulga de forma innecesaria:

- La versión exacta y parche acumulativo de SQL Server 2022 instalado.
- El número exacto de procedimientos almacenados, triggers, vistas y funciones del sistema.
- El mensaje de excepción crudo del driver TDS en caso de pérdida de conexión.

5. Respuesta al Informe Técnico de Respaldos y Contraste de Pruebas

5.1. El Fallo de las Pruebas E2E: Diagnóstico Honesto

Uno de los puntos más críticos del segundo informe de Samir fue la imposibilidad de reproducir el resultado de las pruebas de integración E2E, documentando textualmente:

> **Resultado obtenido por Samir:** `E2E backend: FALLO (2 pruebas fallidas: Login failed for user 'sa')`.
> *"La afirmación de E2E al 100% no se reproduce con la configuración actual: ambas pruebas fallan durante la inicialización del pool SQL por rechazo del usuario sa."*

Al inspeccionar el archivo de pruebas `backend/test/app.e2e-spec.ts` (líneas 1 a 17 y 28 a 39), se identificó el origen exacto del fallo:

```typescript
// Fragmento real de backend/test/app.e2e-spec.ts
class MockConfigService {
  get(key: string) {
    if (key === 'database.server') return 'localhost';
    if (key === 'database.name') return 'MUNDO_ESCOLAR';
    if (key === 'database.user') return 'sa';
    if (key === 'database.password') return 'MundoEscolar2026*';
    if (key === 'database.port') return 1433;
    return null;
  }
}
```

Explicación técnica: En el entorno de desarrollo original, la instancia local de SQL Server 2022 estaba configurada con la contraseña `MundoEscolar2026*` para el login `sa`. Cuando las pruebas se ejecutaban en esa máquina, el pool de conexiones se conectaba con éxito y las dos aserciones HTTP pasaban con código 200.

Sin embargo, en la máquina de Samir (y en cualquier entorno de integración continua o máquina de otro desarrollador), la contraseña de `sa` es distinta o el login `sa` se encuentra deshabilitado por directivas de seguridad de Windows. En consecuencia, el test intentaba autenticarse contra el motor y era rechazado inmediatamente con error `ELOGIN: Login failed for user 'sa'`.

> **Lección de ingeniería aprendida:** Haber certificado "100% de pruebas E2E aprobadas" fue una falla metodológica de nuestra parte al asumir que la configuración de una máquina local representaba la portabilidad del código. **Samir tiene la razón completa.** Las pruebas deben configurarse para usar variables de entorno dinámicas o una base de datos de pruebas aislada.

5.2. Auditoría de Dependencias de Producción (npm audit)

Samir ejecutó `npm audit --omit=dev` identificando 7 vulnerabilidades de producción en el backend. La verificación realizada en esta sesión arrojó exactamente las mismas 7 vulnerabilidades (5 altas, 2 moderadas):

- **multer (Alta - GHSA-q7qp-rjg7-q26m):** Denegación de servicio mediante operaciones multipart asíncronas no liberadas. Dependencia de `@nestjs/platform-express`.
- **body-parser (Alta - GHSA-27r3-jxmh-p828):** DoS mediante bytes residuales tras descompresión de peticiones maliciosas.
- **@azure/msal-node (Alta - GHSA-xcx6-jx8p-x3h5):** Potencial divulgación de parámetros de autorización hacia endpoints controlados por atacantes. Dependencia indirecta de `@azure/identity` requerida por el driver `mssql`.
- **js-yaml (Moderada - GHSA-r3ph-w7gj-g6xm):** Consumo excesivo de CPU mediante claves merge no delimitadas. Dependencia de `@nestjs/swagger`.

Se ratifica el dictamen de Samir: **estas dependencias deben remediarse de forma controlada** mediante `npm audit fix` y actualización de parches compatibles sin generar cambios disruptivos en la compilación de NestJS.

6. Arquitectura de Remediación Definitiva (Cierre de Hallazgos)

Para transformar el prototipo en una solución verdaderamente robusta y defendible ante el profesor Torrentes, se establece la siguiente matriz de compromisos técnicos inmediatos:

PRIORIDAD	COMPONENTE / ARCHIVO	ACCIÓN TÉCNICA DE REMEDIACIÓN	CRITERIO DE CIERRE VERIFICABLE
P0	**Procedimientos DBA** `10_CrearProcedimientos.sql` (SPs 69 a 74)	**Eliminar SQL Dinámico Abierto y Desactivar TRUSTWORTHY:** 1. Reemplazar concatenación de rutas por validación estricta de nombres canónicos. 2. Confinar la ruta al directorio canónico `C:SQLBackup`. 3. Ejecutar `ALTER DATABASE MUNDO_ESCOLAR SET TRUSTWORTHY OFF;`.	Inyección SQL neutralizada; rechazo de comillas simples, traversals (`..`) y rutas UNC. Base opera con `TRUSTWORTHY OFF`.
P0	**Registro de Clientes** `clientes.service.ts` `clientes.dto.ts`	**Eliminar Contraseña Predeterminada:** 1. Eliminar fallback `'Cliente2026!'`. 2. Establecer `@IsNotEmpty()` y `@MinLength(8)` en DTOs de cliente particular e institucional.	Peticiones de registro sin contraseña son rechazadas con error HTTP 400 Bad Request. Cero cuentas creadas con claves comunes.
P0	**Pruebas E2E** `app.e2e-spec.ts`	**Desacoplar Credenciales Hardcoded:** Configurar las pruebas E2E para consumir credenciales de entorno (`process.env.DB_PASSWORD`) o mockear la capa de datos.	`npm run test:e2e` ejecuta y aprueba al 100% (2/2 suites) en cualquier máquina sin requerir clave fija de `sa`.
P1	**Segregación PoLP** `Seguridad.sql`	**Retirar db_backupoperator de App:** 1. Remover `usr_mundo_app` del rol `db_backupoperator`. 2. Revocar `GRANT EXECUTE ON SCHEMA::dbo` y otorgar permisos solo a los SPs de negocio estrictamente requeridos.	La cuenta técnica de la app no puede ejecutar comandos administrativos directos ni respaldar fuera de los SPs autorizados.
P1	**Filtro de Excepciones** `sql-exception.filter.ts` `sql.service.ts`	**Sanitización de Respuestas de Error:** No enviar `exception.message` ni metadatos SQL crudos al cliente. Generar código de error opaco y registrar el detalle en logs.	Las respuestas HTTP nunca contienen nombres de tablas, columnas ni versiones internas del motor SQL.
P1	**Control de Abuso** `app.module.ts` `auth.controller.ts`	**Implementar Rate Limiting (Throttler):** Instalar `@nestjs/throttler` y aplicar guardias de tasa (10 req/min) sobre autenticación y registro.	Pruebas automatizadas demuestran respuesta HTTP 429 Too Many Requests ante intentos repetidos de acceso.

7. Conclusión y Compromiso de Calidad

La auditoría presentada por Samir representa un aporte de inmenso valor técnico para el proyecto Mundo Escolar. Lejos de ser una confrontación personal, **sus observaciones pusieron al descubierto vulnerabilidades reales de arquitectura que habrían puesto en grave peligro la calificación final ante el profesor Torrentes**.

Con este dictamen, el equipo de desarrollo asume con madurez técnica cada una de las correcciones priorizadas, dando inicio inmediato a la fase de remediación de los puntos P0 y P1 para consolidar un sistema seguro, defendible y alineado con los más altos estándares de ingeniería de bases de datos.